

Area Aktif & Ingatan Cepat

Window yang bergerak, pointer yang menyempit, akumulasi masa lalu, dan memori instan.

Tujuan dokumen ini: bukan menghafal tanda-tanda soal, tapi memahami gerak berpikir pattern. Setiap pattern dijelaskan dengan contoh dunia nyata, contoh teknologi, konsep komputasi, alasan kenapa contoh itu sama dengan pattern, dan template Dart minimal.

Cara Membaca Dokumen Ini

Jangan baca seperti kamus keyword. Baca seperti latihan membangun intuisi. Pattern LeetCode itu mirip alat kerja: tiap alat punya cara bergerak, situasi natural, dan bentuk data yang cocok.

Untuk tiap pattern, pakai urutan baca ini: makna manusiawi -> contoh dunia nyata -> contoh teknologi -> konsep komputasi -> bentuk LeetCode -> template Dart -> pertanyaan cek pemahaman.

Pattern	Makna inti
Sliding Window	menjaga area aktif yang bergerak
Two Pointers	dua posisi bergerak untuk mengurangi pencarian
Prefix Sum	membawa akumulasi masa lalu
HashMap	ingatan cepat dari key ke value
HashSet	ingatan cepat apakah sesuatu pernah muncul

Sliding Window - menjaga area aktif yang bergerak

Makna manusiawinya

Sliding window berarti lu hanya peduli pada rentang aktif saat ini. Rentang itu bisa membesar, lalu mengecil kalau melanggar aturan. Fokusnya bukan semua data, tapi area yang sedang relevan.

Contoh nyata non-teknologi

Pantau pengeluaran 7 hari terakhir

Setiap hari data baru masuk, data hari ke-8 yang terlalu lama keluar. Lu tidak peduli pengeluaran sejak lahir, hanya jendela 7 hari terakhir.

Melihat performa olahraga mingguan

Lu track rata-rata latihan 4 minggu terakhir. Minggu baru masuk, minggu lama keluar.

Contoh nyata di dunia teknologi

Analytics real-time

Dashboard menampilkan request per 5 menit terakhir. Window bergerak terus mengikuti waktu sekarang.

Rate limiter

User boleh 100 request per menit. Sistem menjaga window request aktif dan menghapus request yang sudah lewat.

Konsep komputasinya

Secara komputasi, sliding window memakai dua batas: left dan right. Right menambah elemen baru. Left membuang elemen lama sampai window valid lagi. State disimpan untuk isi window: count, sum, unique, max, dan sebagainya.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena ada rentang aktif yang berubah seiring waktu/posisi. Kita menghindari hitung ulang dari nol dengan mengupdate saat elemen masuk dan keluar.

Bentuknya di LeetCode

- Longest/shortest subarray atau substring contiguous
- At most/at least K
- Window sum, frequency, unique chars
- Minimum Window Substring, Longest Substring Without Repeating Characters

Template Dart minimal

```
int longestUniqueSubstring(String s) {  
  final count = <String, int>{};  
  int left = 0;  
  int best = 0;  
  
  for (int right = 0; right < s.length; right++) {  
    final ch = s[right];  
    count[ch] = (count[ch] ?? 0) + 1;  
  
    while (count[ch]! > 1) {  
      final leftCh = s[left];  

```

```
        count[leftCh] = count[leftCh]! - 1;
        left++;
    }
    best = best > right - left + 1 ? best : right - left + 1;
}
return best;
}
```

Kalau masih bingung, tanya begini

- Apa isi window?
- Kapan window valid?
- Kapan left harus maju?
- Apa yang terjadi saat elemen keluar?

Mini drill pemahaman

1. Bikin analogi sliding window untuk monitoring CPU 10 menit terakhir.
2. Kenapa sliding window butuh contiguous?
3. Kapan sliding window gagal dan prefix sum lebih cocok?

Two Pointers - dua posisi bergerak untuk mengurangi pencarian

Makna manusiawinya

Two pointers menggunakan dua posisi untuk membuat pencarian lebih hemat. Dua posisi ini bisa dari kiri-kanan, slow-fast, atau reader-writer. Intinya: jangan cek semua pasangan kalau arah geraknya bisa memberi informasi.

Contoh nyata non-teknologi

Mencari dua orang dengan tinggi total tertentu

Daftar orang sudah diurutkan tinggi. Kalau pasangan terlalu pendek, naikan orang pendek. Kalau terlalu tinggi, turunkan orang tinggi. Lu tidak coba semua pasangan.

Merapikan barang dari dua sisi

Lu mau memindahkan barang buruk ke belakang dan barang baik ke depan. Satu pointer scan, satu pointer menandai posisi taruh.

Contoh nyata di dunia teknologi

Dedup sorted array

Satu pointer membaca semua data, satu pointer menulis posisi unik berikutnya.

Palindrome check

Pointer kiri dan kanan membandingkan karakter luar ke dalam.

Konsep komputasinya

Secara komputasi, two pointers memanfaatkan struktur data, sering sorted order atau constraint in-place. Gerak pointer harus punya alasan: kalau sum kecil, left naik; kalau sum besar, right turun.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena dua posisi memberi informasi untuk membuang banyak kemungkinan sekaligus. Kita tidak brute force, melainkan mempersempit area pencarian.

Bentuknya di LeetCode

- Two Sum II, 3Sum
- Valid Palindrome
- Remove duplicates in-place
- Container With Most Water
- Fast-slow linked list

Template Dart minimal

```
List<int> twoSumSorted(List<int> nums, int target) {  
  int left = 0;  
  int right = nums.length - 1;  
  
  while (left < right) {  
    final sum = nums[left] + nums[right];  
    if (sum == target) return [left, right];  
  }  
}
```

```
        if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
    return [-1, -1];
}
```

Kalau masih bingung, tanya begini

- Apa arti pointer kiri dan kanan?
- Apa alasan pointer ini bergerak?
- Apakah data sorted atau punya arah?
- Apakah gue mengurangi ruang pencarian?

Mini drill pemahaman

1. Kenapa sorted array membuat two pointers valid?
2. Apa beda two pointers dan sliding window?
3. Bikin contoh fast-slow pointer di kehidupan nyata.

Prefix Sum - membawa akumulasi masa lalu

Makna manusiawinya

Prefix sum berarti menyimpan total kumulatif agar pertanyaan tentang rentang tidak perlu dihitung ulang. Ini seperti membawa catatan jumlah dari masa lalu.

Contoh nyata non-teknologi

Catatan pengeluaran bulanan

Kalau lu punya total kumulatif sampai tiap tanggal, total tanggal 5-12 cukup total sampai 12 dikurangi total sampai 4.

Meteran listrik

Angka meteran sekarang dikurangi angka meteran bulan lalu memberi pemakaian periode itu.

Contoh nyata di dunia teknologi

Log analytics

Jumlah request antara jam A dan B bisa dihitung dari cumulative count.

Database/reporting

Precomputed aggregates mempercepat query range.

Konsep komputasinya

Secara komputasi, `prefix[i]` menyimpan total sebelum/sampai posisi `i`. Range sum menjadi $O(1)$. Untuk subarray sum tertentu, prefix sum sering dipasangkan dengan `HashMap`.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena semua memakai selisih dua akumulasi untuk mengetahui nilai di antara dua titik. Masa lalu disimpan agar masa kini tidak menghitung ulang.

Bentuknya di LeetCode

- Range Sum Query
- Subarray Sum Equals K
- Find Pivot Index
- Contiguous Array
- Array dengan angka negatif yang tidak cocok sliding window

Template Dart minimal

```
class PrefixSum {
  final List<int> prefix;

  PrefixSum(List<int> nums) : prefix = List.filled(nums.length + 1, 0) {
    for (int i = 0; i < nums.length; i++) {
      prefix[i + 1] = prefix[i] + nums[i];
    }
  }

  int rangeSum(int left, int right) {
    return prefix[right + 1] - prefix[left];
  }
}
```

}

Kalau masih bingung, tanya begini

- Apa yang gue akumulasikan?
- Query range apa yang ingin dipercepat?
- Apakah angka negatif membuat sliding window tidak valid?
- Perlukah HashMap untuk prefix yang pernah muncul?

Mini drill pemahaman

1. Hitung total pengeluaran tanggal 10-20 pakai prefix.
2. Kenapa prefix sum bisa $O(1)$ untuk range sum?
3. Kenapa subarray dengan negatif sering pakai prefix + map?

HashMap - ingatan cepat dari key ke value

Makna manusiawinya

HashMap adalah memori cepat berbasis key. Lu memberi key, langsung dapat value. Ini cocok saat masalahnya bukan urutan, tapi lookup cepat.

Contoh nyata non-teknologi

Buku kontak

Nama orang menjadi key, nomor telepon menjadi value. Lu tidak membaca semua kontak satu-satu setiap kali butuh nomor.

Loker bernomor

Nomor loker adalah key, isi loker adalah value. Kalau tahu nomor, langsung akses.

Contoh nyata di dunia teknologi

Cache API

URL/request key dipakai untuk mengambil response yang sudah disimpan.

Index database

Index membantu menemukan data berdasarkan key tanpa scan semua row.

Konsep komputasinya

Secara komputasi, HashMap memberi rata-rata $O(1)$ untuk insert, lookup, update. Ia sering dipakai untuk frequency, mapping value ke index, complement, grouping, dan memoization.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena semuanya mengubah pencarian lambat menjadi akses langsung. HashMap adalah cara membuat ingatan eksplisit yang bisa ditanya cepat.

Bentuknya di LeetCode

- Two Sum
- Group Anagrams
- Frequency counter
- Subarray Sum Equals K dengan prefix map
- LRU Cache sebagai HashMap + linked list

Template Dart minimal

```
List<int> twoSum(List<int> nums, int target) {  
    final indexByValue = <int, int>{};  
  
    for (int i = 0; i < nums.length; i++) {  
        final need = target - nums[i];  
        if (indexByValue.containsKey(need)) {  
            return [indexByValue[need]!, i];  
        }  
        indexByValue[nums[i]] = i;  
    }  
    return [-1, -1];  
}
```

Kalau masih bingung, tanya begini

- Apa key-nya?
- Apa value yang perlu disimpan?
- Apakah gue menghindari loop ulang?
- Apakah update value diperlukan?

Mini drill pemahaman

1. Di Two Sum, key/value-nya apa?
2. Kenapa frequency char cocok pakai HashMap?
3. Kapan HashMap tidak perlu karena array index cukup?

HashSet - ingatan cepat apakah sesuatu pernah muncul

Makna manusiawinya

HashSet adalah versi lebih sederhana dari HashMap: lu tidak butuh value, cuma butuh tahu apakah item sudah ada. Ia adalah daftar pernah-muncul yang cepat.

Contoh nyata non-teknologi

Daftar tamu yang sudah check-in

Kalau nama sudah ada, jangan check-in dua kali.

Stempel tangan di konser

Stempel menandakan orang sudah masuk. Tidak perlu menyimpan detail panjang.

Contoh nyata di dunia teknologi

Visited node

Dalam graph traversal, HashSet mencegah node diproses berkali-kali.

Dedup data

Menghapus email/user id yang duplicate sebelum diproses.

Konsep komputasinya

Secara komputasi, HashSet menyimpan elemen unik dengan lookup cepat. Ia dipakai untuk duplicate detection, membership test, visited, dan uniqueness.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena pertanyaannya boolean: pernah ada atau belum? Bukan: nilainya apa?

Bentuknya di LeetCode

- Contains Duplicate
- Longest Consecutive Sequence
- Visited set BFS/DFS
- Intersection of arrays

Template Dart minimal

```
bool hasDuplicate(List<int> nums) {  
  final seen = <int>{};  
  for (final x in nums) {  
    if (seen.contains(x)) return true;  
    seen.add(x);  
  }  
  return false;  
}
```

Kalau masih bingung, tanya begini

- Apakah gue cuma butuh tahu ada/tidak?
- Apakah urutan tidak penting?
- Apakah duplicate harus dicegah?

Mini drill pemahaman

1. Bedakan kapan pakai HashMap vs HashSet.
2. Kenapa visited di graph harus Set?
3. Apa risiko kalau HashSet tidak dipakai di DFS graph?

Penutup

Mastery bukan berarti menghafal semua tanda soal. Mastery berarti saat melihat masalah, lu bisa memodelkan gerak masalahnya: apakah ia menyebar, menyelam, menjaga window, mencari boundary, mengingat key, menggabungkan kelompok, atau menyimpan subproblem.

Cara latihan: ambil satu pattern, pahami analoginya, kerjakan 3-5 soal mudah, tulis ulang konsepnya dengan bahasa sendiri, lalu campur dengan pattern lain supaya otak belajar mengenali bentuk masalah tanpa spoiler.